

Exp 5

1. Aim

To study the basic structure and constructs of the Solidity programming language and to create, compile, deploy, and execute smart contracts using the Remix IDE and Remix VM.

2. Tasks Performed

The following tasks were performed during the experiment:

1. Opened and explored the Remix IDE.
2. Explored the Solidity Basics LearnETH course available in Remix IDE.
3. Studied the structure of a Solidity smart contract.
4. Created Solidity programs using:
 - Primitive data types and variables
 - Functions
 - **view** and **pure** functions
 - Function inputs and outputs
 - Visibility and modifiers
 - Constructors
 - **if-else** statements and loops
 - Arrays
 - Mappings
 - Structures and enums
 - Data locations
5. Studied blockchain transaction concepts such as:
 - Ether and Wei
 - Gas and gas price
 - Sending transactions
6. Compiled Solidity programs using the Solidity Compiler in Remix IDE.
7. Deployed smart contracts using the Remix VM.
8. Executed the deployed contract functions and verified their outputs.
9. Completed the 19 assignments provided in the Solidity Basics LearnETH course.

Program Code and Output:

Tutorial -01 Introduction

REMIX 2.5.5 Learneth Solidity Beginner Course

SOLIDITY COMPILER

COMPILER + 0.8.34+commit.80d5c536

☐ Include nightly builds

☐ Auto compile

☐ Hide warnings

Advanced Configurations

[Compile introduction.sol](#)

[Compile and Run script](#)

CONTRACT

Counter (introduction.sol)

[Run Remix Analysis](#)

[Run SolidityScan](#)

[Publish on IPFS](#)

[Publish on Swarm](#)

[Create a DApp with your contracts](#) [Start now](#)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9
10
11
12     // Function to increment count by 1
13     function inc() public { infinite gas
14         count += 1;
15     }
16
17     // Function to decrement count by 1
18     function dec() public { infinite gas
19         count -= 1;
20     }
21 }
```

Ctrl+S to compile 1. Introduction/introduction.sol

[Explain contract](#) AI copilot

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

DEPLOY & RUN TRANSACTIONS

Environment [Fork](#) [Reset](#)

Remix VM Osaka

Account 1 0x5B3...eddC4 100.000 ETH

Deploy [Remix VM osaka](#)

Counter introduction.sol [Compiled](#)

Value 0 wei

Gas limit auto 0

[Deploy](#)

[Create a DApp with your contracts](#) [Start now](#)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10     }
11
12     // Function to increment count by 1
13     function inc() public { infinite gas
14         count += 1;
15     }
16
17     // Function to decrement count by 1
18     function dec() public { infinite gas
19         count -= 1;
20     }
21 }
```

[Explain contract](#)

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

get :

CALL [call] from: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c Debug

from	0x5838Da6a701c568545dCfcB03FcB875f56beddC4
to	Counter.get() 0xd9145CCE52D386f254917e481e844e9943F39138
execution cost	2453 gas (Cost only applies when called by a contract)
input	0x6d4...ce63c
output	0x00
decoded input	{}
decoded output	{ "0": "uint256: 0" }

inc:

✓ [vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x571...305C0 logs: 0 hash: 0xbd5...04b2b Debug

status	1 Transaction mined and execution completed
transaction hash	0xbd586afadc5f2f1743526c64b6f34b44d70441bfd9211278003bf9c1c5204b2b
block hash	0xe9943769c8719871abcb2804309fa534aae966626b1dbd63c0ed60ccaef35074c
block number	3
from	0x5838Da6a701c568545dCfcB03FcB875f56beddC4
to	Counter.inc() 0xd9145CCE52D386f254917e481e844e9943F39138
transaction cost	43517 gas
execution cost	22453 gas

dec:

✓ [vm] from: 0x583...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xf87...5fdc7 Debug

status	1 Transaction mined and execution completed
transaction hash	0xf87421f6a66302c609d1cf9cad54c1a4e8155f9b38d2025580e167a180c5fdc7
block hash	0xb28ba7076100186710c49f396c112a48cbc25d037a8f1862bd27745f6449bf0d
block number	4
from	0x5838Da6a701c568545dCfcB03FcB875f56beddC4
to	Counter.dec() 0xd9145CCE52D386f254917e481e844e9943F39138
transaction cost	21661 gas

Tutorial-02 Basic Syntax:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

Tutorials list 2. Basic Syntax 2 / 19

Don't worry if you didn't understand some concepts like **variety**, **data types**, or **state variables**. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ **Assignment**

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts Start now

```

1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }

```

Explain contract AI copilot

0 ☐ Listen on all transactions

from	0x5B38D6a701c56854dCfC803fC8875F56beddC4
to	Counter.dec() 0xd9145CCe520386F254917e481e844e9943F30138
transaction cost	21661 gas

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial-03 Primitive Data Types:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

Tutorials list 3. Primitive Data Types 3 / 19

and **structs**

[Watch a video tutorial on Primitive Data Types.](#)

★ **Assignment**

1. Create a new variable `newAddr` that is a `public address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts Start now

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Primitives {
5     bool public boo = true;
6
7     /*
8      * uint stands for unsigned integer, meaning non negative integers
9      * different sizes are available
10      *   uint8   ranges from 0 to 2 ** 8 - 1
11      *   uint16  ranges from 0 to 2 ** 16 - 1
12      *   ...
13      *   uint256 ranges from 0 to 2 ** 256 - 1
14      */
15     uint8 public u8 = 1;
16     uint public u256 = 456;
17     uint public u = 123; // uint is an alias for uint256
18
19     /*
20      * Negative numbers are allowed for int types.
21      * Like uint, different ranges are available from int8 to int256

```

Explain contract AI copilot

0 ☐ Listen on all transactions

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial-04 Variables:

LEARNETH

4. Variables
4 / 19

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ **Assignment**

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts Start now

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }

```

[Explain contract](#) AI copilot

0 ☐ Listen on all transactions

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial-5.1 Reading and Writing to a State Variable :

LEARNETH

5.1 Functions - Reading and Writing to a State Variable
5 / 19

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ **Assignment**

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts Start now

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9
10    // You need to send a transaction to write to a state variable.
11    function set(uint _num) public { 22536 gas
12        num = _num;
13    }
14
15    // You can read from a state variable without sending a transaction.
16    function get() public view returns (uint) { 2475 gas
17        return num;
18    }
19
20    function get_b() public view returns (bool) { 2539 gas
21        return b;
22    }
23 }

```

[Explain contract](#) AI copilot

0 ☐ Listen on all transactions

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial-5.2 View and Pure:

REMIX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH

5.2 Functions - View and Pure (6 / 19)

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function add(uint i, uint j) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addToX2(uint y) public {
18        x = x + y;
19    }
20 }
```

Tutorial-5.3 Functions - Modifiers and Constructors:

REMIX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH

5.3 Functions - Modifiers and Constructors (7 / 19)

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract FunctionModifier {
5     // We will use these variables to demonstrate how to use
6     // modifiers.
7     address public owner;
8     uint public x = 10;
9     bool public locked;
10
11     constructor() {
12         // Set the transaction sender as the owner of the contract.
13         owner = msg.sender;
14     }
15
16     // Modifier to
17     // the contract
18     modifier onlyOwner {
19         require(msg.sender == owner, "Not owner");
20         // Underscore is a special character only used inside
21         // a function modifier and it tells Solidity to
```

Tutorial-5.4 Functions - Inputs and Outputs:

REMIX 2.5.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.4 Functions - Inputs and Outputs 8 / 19

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer Show answer

Next

Well done! No errors.

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

RemixAI Copilot (enabled)

```

72
73 // Can use array for output
74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) {
77     return arr;
78 }
79
80 function returnTwo()
81     public
82     pure
83     returns (
84         int i,
85         bool b
86     )
87 {
88     i = -2;
89     b = true;
90 }
91

```

Tutorial-06 Visibility:

REMIX 2.5.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

6. Visibility 9 / 19

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer Show answer

Next

Well done! No errors.

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

RemixAI Copilot (enabled)

```

51 // State variables cannot be external so this code won't compile.
52 // string external externalVar = "my external variable";
53 }
54
55 contract Child is Base {
56     // Inherited contracts do not have access to private functions
57     // and state variables.
58     // function testPrivateFunc() public pure returns (string memory) {
59     //     return privateFunc();
60     // }
61
62     // Internal function call be called inside child contracts.
63     function testInternalFunc() public pure override returns (string memory) {
64         return internalFunc();
65     }
66
67     function testInternalVar() public view returns (string memory, string memory) {
68         return (internalVar, publicVar);
69     }
70 }

```

Tutorial-7.1 Control Flow- If/Else:

REMIX2.5.5Learneth Solidity Beginner Course

LinkSee PlansGet AI Credits

LEARNETH

Tutorials list

Syllabus

7.1 Control Flow - If/Else10 / 19

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
9      return 1;
10     } else {
11         return 2;
12     }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // return 2;
20
21     // shorthand way to write if / else statement
22     return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck(uint y) public pure returns (bool) {
26     return y%2 == 0 ? true : false;
27 }
28 }
```

Explain contract

AI copilot

0Listen on all transactionsFilter with transaction hash or address

Scam AlertInitialize as git repoDid you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.RemixAI Copilot (enabled)

Tutorial-7.2 Control Flow- Loops:

REMIX2.5.5Learneth Solidity Beginner Course

LinkSee PlansGet AI Credits

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops11 / 19

prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

- Create a public `uint` state variable called count in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check AnswerShow answer

Next

Well done! No errors.

Create a DApp with your contractsStart now

```
8      for (uint i = 0; i < 10; i++) {
9          if (i == 5) {
10             // Skip to next iteration with continue
11             continue;
12          }
13          if (i == 5) {
14             // Exit loop with break
15             break;
16          }
17          count++;
18      }
19
20      // while loop
21      uint j;
22      while (j < 10) {
23          j++;
24      }
25  }
26
27 }
```

Explain contract

AI copilot

0Listen on all transactionsFilter with transaction hash or address

Scam AlertInitialize as git repoDid you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.RemixAI Copilot (enabled)

Tutorial-8.1 Data Structures- Arrays :

LEARNETH 8.1 Data Structures - Arrays 12 / 19

element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

[Check Answer](#) [Show answer](#)

Well done! No errors.

```
54 // Move the last element into the place to d
55 arr[index] = arr[arr.length - 1];
56 // Remove the last element
57 arr.pop();
58
59
60 function test() public {
61     arr.push(1);
62     arr.push(2);
63     arr.push(3);
64     arr.push(4);
65     // [1, 2, 3, 4]
66
67     remove(1);
68     // [1, 4, 3]
69
70     remove(2);
71     // [1, 4]
72
73 }
```

Tutorial-8.2 Data Structures- Mappings:

LEARNETH 8.2 Data Structures - Mappings 13 / 19

removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

[Check Answer](#) [Show answer](#)

Well done! No errors.

```
27 mapping(address => mapping(uint
28
29 function get(address _addr1, uint _i) public view returns (bool) {
30     // You can get values from a nested mapping
31     // even when it is not initialized
32     return nested[_addr1][_i];
33 }
34
35 function set(
36     address _addr1,
37     uint _i,
38     bool _boo
39 ) public {
40     nested[_addr1][_i] = _boo;
41 }
42
43 function remove(address _addr1, uint _i) public {
44     delete nested[_addr1][_i];
45 }
46 }
```

Tutorial-8.3 Data Structures- Structs:

REMIX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

8.3 Data Structures - Structs | 14 / 19

member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts | Learneth Solidity Beginner Course/7.2 | Control Flow - Loops/loops.sol

```

32   Todo storage
33   return (todo.text, todo.completed);
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public {
38   Todo storage todo = todos[_index];
39   todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public {
44   Todo storage todo = todos[_index];
45   todo.completed = !todo.completed;
46 }
47
48 function remove(uint _index) public {
49   delete todos[_index];
50 }
51 }

```

[Explain contract](#) | AI copilot ☒

0 ☐ Listen on all transactions | Filter with transaction hash or address

Scam Alert | Initialize as git repo | Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. | RemixAI Copilot (enabled)

Tutorial-8.4 Data Structures- Enums:

REMIX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

8.4 Data Structures - Enums | 15 / 19

and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts | Start now

```

28
29 function getSize() public view returns (Size) {
30   return sizes;
31 }
32
33 // Update status by passing uint into input
34 function set(Status _status) public {
35   status = _status;
36 }
37
38 // You can update to a specific enum like this
39 function cancel() public {
40   status = Status.Canceled;
41 }
42
43 // delete resets the enum to its first value, 0
44 function reset() public {
45   delete status;
46 }
47 }

```

[Explain contract](#) | AI copilot ☒

0 ☐ Listen on all transactions | Filter with transaction hash or address

Scam Alert | Initialize as git repo | Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. | RemixAI Copilot (enabled)

Tutorial-09 Data Locations:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

9. Data Locations 16 / 19

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer **Show answer**

Next

Well done! No errors.

dataLocations.sol

```

28 function f( /* undefined gas
29     uint[] storage _arr,
30     mapping(uint => address) storage _map,
31     MyStruct storage _myStruct
32 ) internal {
33     // do something with storage variables
34 }
35
36 // You can return memory variables
37 function g(uint[] memory _arr) public returns (uint[] memory) { /* infinite gas
38     // do something with memory array
39     _arr[0] = 1;
40 }
41
42 function h(uint[] calldata _arr) external { /* 468 gas
43     // do something with calldata array
44     _arr[0] = 1;
45 }
46 }
47

```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial-10.1 Transactions- Ether and Wei

LEARNETH 2.5.5 Learneth Solidity Beginner Course

10.1 Transactions - Ether and Wei 17 / 19

Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer **Show answer**

Next

Well done! No errors.

etherAndWei.sol

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }

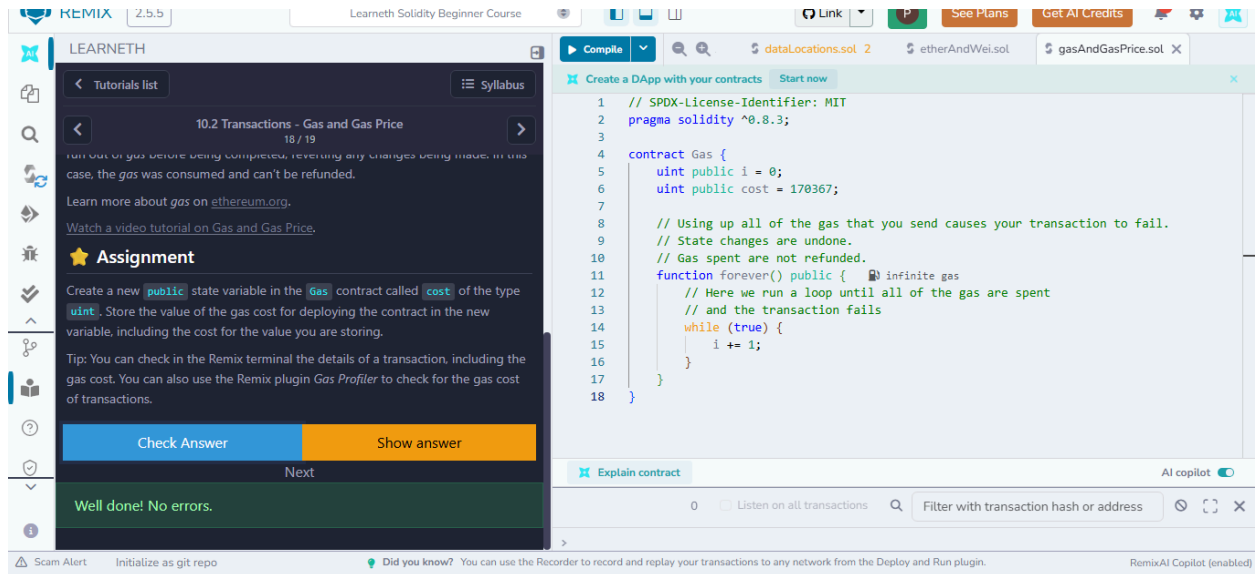
```

Explain contract AI copilot

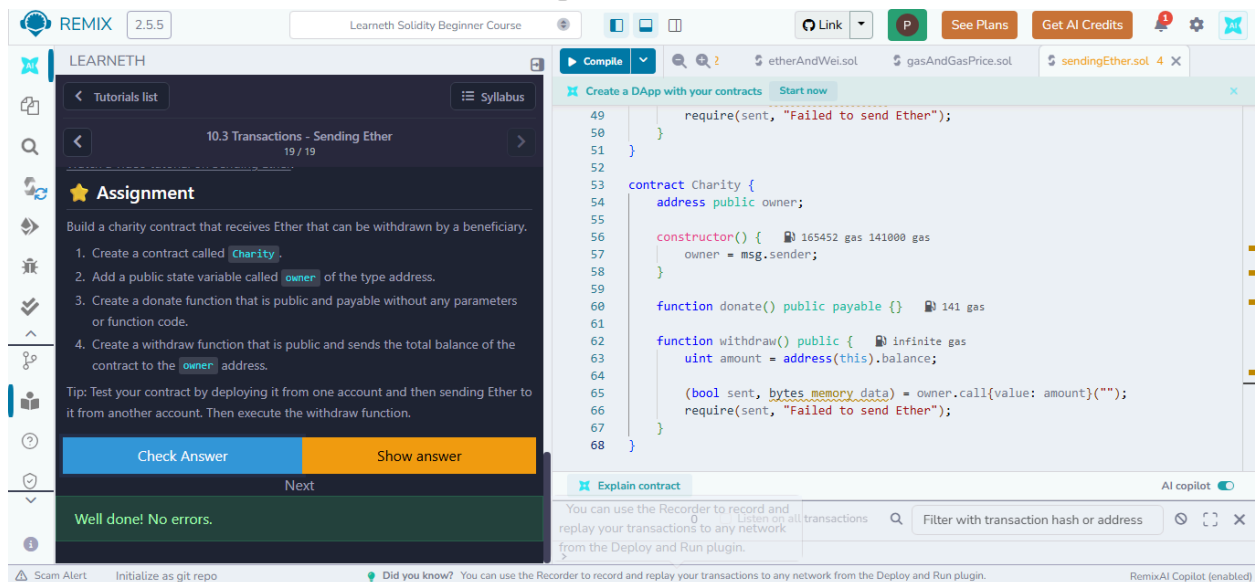
0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial-10.2 Transactions- Gas and Gas Price:



Tutorial-10.3 Transactions- Sending Ether:



5. Conclusion

The experiment was successfully completed using Remix IDE. The basic structure and constructs of Solidity were studied, and smart contracts were successfully written, compiled, deployed, and executed using Remix VM. The experiment provided practical knowledge of Solidity programming, smart contract development, and basic blockchain transaction concepts. The 19 assignments from the Solidity Basics LearnETH course were also completed successfully.